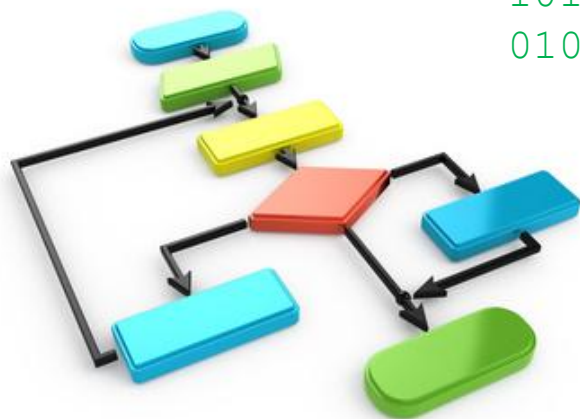


μControladores II

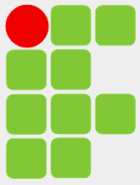
Técnicas de Programação



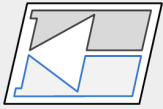
```
101001000110101010  
010001001001100001
```



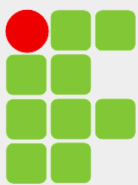
Prof. Charles Borges de Lima.



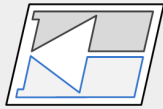
SUMÁRIO



- Introdução
- Depuração
- Melhorando a Qualidade do Código
- Otimização
- Algoritmos com FSM
- *Watchdog Timer*
- Conclusão



INTRODUÇÃO

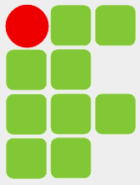


Programar exige prática, conhecimento técnico, organização e estudo constante.

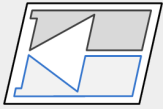
Para se obter o máximo de desempenho é fundamental conhecer o hardware que se está programando. Da mesma forma, conhecer como o compilador monta o código ajuda nas otimizações.

Não existem fórmulas mágicas, muito se aprende copiando, observando os programadores mais experientes, escrevendo códigos e depurando.

Códigos legíveis nem sempre são bons códigos e bons códigos nem sempre são legíveis. Um bom algoritmo é importante, assim como a documentação.



Depuração



Uma grande parcela do tempo gasto no desenvolvimento de um programa é no seu teste e depuração, principalmente em sistemas embarcados.

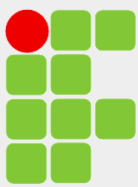
Use truques no programa para depuração:

- *Prints* em pontos estratégicos (podem ser comentados mais tarde)
- Envio de dados seriais para verificação em algum terminal virtual.

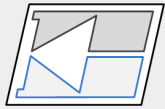
Use o HW para a depuração:

- Usar sempre que possível um LED para sinalizar alguma operação de teste.
- Use alguns pinos de I/O para ver ações do programa (analisador de sinais) – **muito importante!**
- Coloque pontos de análise de sinais na PCI, incluindo VCC e GND.

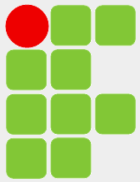
Um bom projeto funciona, um grande projeto também é fácil de depurar.



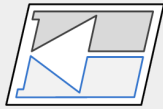
Melhorando a Qualidade do Código



- Escreva o código com algum guia de estilo (*Style Guide**). A padronização é muito importante.
- Números de 16 bits expresso com 16 bits (**0x00AE** ao invés de **0xAE**)
- Empregar verbos nos nomes de funções, com nomes facilmente identificáveis (`init_canal()`; `get_error_code()`).
- Não use letras como separador `InitCanal()`, `getErrorCode()`. É + difícil de ler, o mesmo vale para nomes de variáveis. Quanto mais legível o código melhor.
- Evite usar variáveis globais, são extremamente problemáticas, muito difíceis de depurar e apresentam problemas de reentrância.
- O uso de variáveis globais deve ser aprovada pelo gerente do projeto.



Melhorando a Qualidade do Código



O código deve ser comentado durante a escrita, não posteriormente. Empregue comentários que geram documentação automática através de alguma ferramenta (ex. Doxygen).

Use ferramentas de desenvolvimento adequadas:

- Controle de versão
- Gerador de documentação
- Gerador de fluxograma
- Depurador (ICE - *In Circuit Emulator*)
- Reuse e não reinvente.

Revise o Código.

Mais usual:

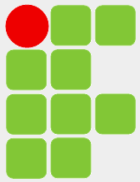
```
statement
{
    statements;
}
```

versus

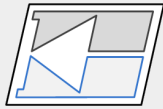
```
statement
{
    statement;
}
```

ou

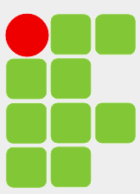
```
statement{
    statements;
}
```



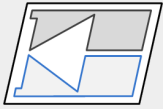
Funções



- Sempre o menor possível, mais de 50 linhas de código é considerado longa.
- Nomes claros e precisos.
- Usar poucos argumentos (≤ 4), muitos são passados pela pilha, degradando o desempenho. *Alinhar os argumentos (menor para maior tamanho).
- Encapsular dados se possível (usar *static* se não houver limitação de memória, desempenho), evitar variáveis globais.
- Remova redundâncias de código, código inútil*.
- Refazer funções que necessitam de muita manutenção.
- Refazer funções confusas/ineficientes.
- Devem, sempre que possível, ser fáceis de interpretar (pelo menos através dos comentários).
- Comente cuidadosamente, de forma concisa. Códigos implementam algoritmos, comentários dizem o funcionamento do código para o próprio programador ou outro.
- Cuidado com as versões.
- Cuidado com os arredondamentos, truncamentos, *overflow* e *underflow*.
- Cuidados com os *warnings* da compilação (variáveis não usadas).

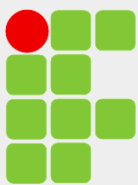


Portabilidade

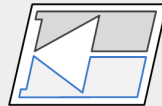


Use ***typedefs*** para a declaração de variáveis, use compiladores compatíveis com o C-99, exemplo:

bits	signed	unsigned
8	<code>int8_t</code>	<code>uint8_t</code>
16	<code>int16_t</code>	<code>uint16_t</code>
32	<code>int32_t</code>	<code>uint32_t</code>
64	<code>int64_t</code>	<code>uint64_t</code>



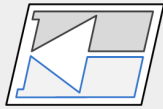
Otimização



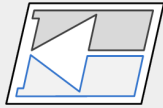
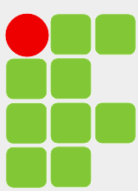
Otimização de código toma tempo e reduz a compreensão do código. Então, é importante otimizar as funções que são executadas frequentemente e importantes para o desempenho do processamento. Entretanto, algumas regras básicas podem ser empregadas sem a necessidade de se recorrer a recursos de otimização elaborados.

Otimizar é questão de prática, do conhecimento do hardware e do software empregados. Só se aprende a otimizar quando se escreve códigos com isso em mente.

Uso Eficiente dos Tipos de Variáveis



- Use `unsigned int` e `signed int` sempre que possível. Use o tipo correspondente à CPU (`uint32_t` para ARM, `uint16_t` para MSP430, `uint8_t` para AVR). No caso do ARM, evite usar variáveis de 8 ou 16 bits, só quando a aritmética com elas é útil (ex. $255+1=0$). Lembre-se que as operações da CPU são efetuadas com o seu tipo de dado nativo.
- Quando houver problema com o uso da memória, use o menor tipo de dado possível.
- Evite *casting* pois eles geralmente custam ciclos extras de processamento. O uso deve ser feito com cuidado, podem ser muito úteis para a leitura de matrizes e variáveis globais para variáveis locais, ou vice-versa (escrita). O *casting* pode melhorar o desempenho, pois as operações podem ser realizada com o tipo de dado nativo do processador.



Organização de Estruturas

- Procure ordenar a declaração de variáveis em ordem crescente (alguns uC tem problemas de alinhamento de dados na memória).
- Coloque todos os elementos de 8 bits no início da estrutura, seguidos pelos de 16, 32 e 64 bits. Coloque os elementos maiores no final da estrutura.

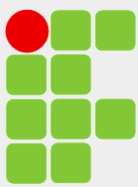
```
struct {  
    char a;  
    int b;  
    char c;  
    short d;  
}
```

Address	+3	+2	+1	+0
+0	pad	pad	pad	a
+4	b[31,24]	b[23,16]	b[15,8]	b[7,0]
+8	d[15,8]	d[7,0]	pad	c

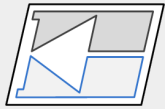
Preenchimento

```
struct {  
    char a;  
    char c;  
    short d;  
    int b;  
}
```

Address	+3	+2	+1	+0
+0	d[15,8]	d[7,0]	c	a
+4	b[31,24]	b[23,16]	b[15,8]	b[7,0]



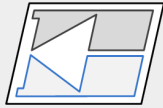
Alocação de Registradores



Os compiladores sempre tentam alocar registradores para variáveis locais, quando isso não é possível elas são colocadas na pilha, o que torna o acesso a elas mais lento.

- Para implementar eficientemente um função o número de variáveis em pilha deve ser minimizado.
- Garanta que as variáveis mais importantes e de acesso frequente sejam armazenadas em registradores (é possível tentar obrigar o compilador usando a palavra *register*)
- Tente limitar o número de variáveis locais nos laços internos de funções.

Uso Eficiente dos Tipos de Variáveis



Exemplos:

```
unsigned int a, b;
```

```
a = a + b;
```

```
ADD r0, r0, r1
```

```
signed short a, b;
```

```
a = a + b;
```

```
ADD r2, r0, r1
```

```
LSL r2, r2, #16
```

```
ASR r0, r2, #16
```

```
short checksum_v4(short *data)
```

```
{  
    unsigned int i;  
    int sum=0;
```

```
    for (i=0; i<64; i++)
```

```
    {  
        sum += *(data++);  
    }
```

```
    return (short)sum;
```

```
}
```

Devido ao pós incremento, disponível nas instruções do ARM. Essa opção é mais eficiente que `sum += data[i]`.

```
short add_v1(short a, short b) <----- Sempre que possível passar
```

```
{  
    return a + (b>>1);  
}
```

e retorna variáveis como 32 bits (int).

```
int checksum_v1(int *data)
```

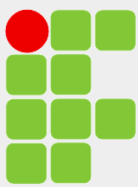
```
{  
    char i; <----- int i  
    int sum=0;
```

```
    for (i=0; i<64; i++)
```

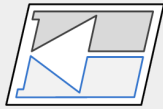
```
    {  
        sum += data[i];  
    }
```

```
    return sum;
```

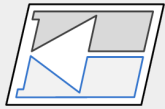
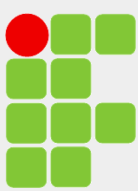
```
}
```



Estruturas de Laços



- Use laços com contagem decrescente até zero. O compilador não necessita alocar um registrador para armazenar o valor de comparação.
- Use contadores sem sinal e a condição `!=0` ao invés de `i>0`. Diminui o tempo de execução do laço (*loop overhead*).
- Use **do-while** versus **for**. Economiza um teste no início.
- Desenrole laços importantes para diminuir o *overhead*.
- Tente organizar o número de elementos de vetores como múltiplos de 4 ou 8, facilitando o *loop unrolling*.



Estruturas de Laços

Exemplos:

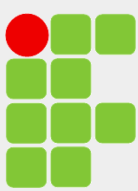
```
int checksum_v6(int *data)
{
    unsigned int i;
    int sum=0;

    for (i=64; i!=0; i--) <---- Utilizar contagem decrescente
    {                                O ARM não precisa analisar i,
        sum += *(data++);           e sim, o flag de status zero.
    }
    return sum;
}
```

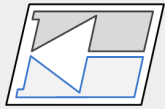


```
int checksum_v8(int *data, unsigned int N)
{
    int sum=0;                      Nr. variável de iterações

    do
    {
        sum += *(data++);
    } while (--N!=0); <---- O ARM não precisa testar
    return sum;           se N=0 na 1ª iteração.
}
```



Estruturas de Laços



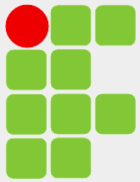
Loop unrolling:

```
int checksum_v9(int *data, unsigned int N)
{
    int sum=0;
    do
    {
        sum += *(data++);
        sum += *(data++);
        sum += *(data++);
        sum += *(data++);
        N -= 4;
    } while ( N!=0);
    return sum;
}
```

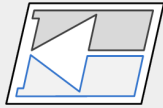
Redução do tempo de execução (Loop overhead)

<---- Repete instruções, diminuindo o nr. de iterações necessárias.

Caso o nr. de iterações não for um múltiplo para um único laço, escrever código para tratar a exceção.



Chamada de Funções

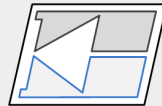
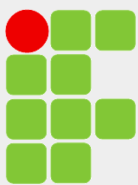


- Tente restringir a quatro o número de argumentos em uma função. Use estruturas para agrupar argumentos relacionados e passe estruturas como ponteiro ao invés de múltiplos argumentos.
- Defina pequenas funções em um mesmo arquivo fonte e antes das funções que as chamam. O compilador pode otimizar a chama de funções e usar *inline*.
- Funções críticas podem ser *inlined* usando a palavra ***_inline***.

Exemplo:

```
char *queue_bytes_v1(      Função com 5 argumentos.
    char *Q_start,         /* Queue buffer start address */
    char *Q_end,           /* Queue buffer end address */
    char *Q_ptr,           /* Current queue pointer position */
    char *data,            /* Data to insert into the queue */
    unsigned int N)        /* Number of bytes to insert */
{
    do
    {
        *(Q_ptr++) = *(data++);

        if (Q_ptr == Q_end)
        {
            Q_ptr = Q_start;
        }
    } while (--N);
    return Q_ptr;
}
```



Chamada de Funções

```
typedef struct {  
    char *Q_start;        /* Queue buffer start address */  
    char *Q_end;          /* Queue buffer end address */  
    char *Q_ptr;          /* Current queue pointer position */  
} Queue;
```

```
void queue_bytes_v2(Queue *queue, char *data, unsigned int N)  
{
```

```
    char *Q_ptr = queue->Q_ptr;  
    char *Q_end = queue->Q_end;
```

```
    do
```

```
    {
```

```
        *(Q_ptr++) = *(data++);
```

```
        if (Q_ptr == Q_end)
```

```
        {
```

```
            Q_ptr = queue->Q_start;
```

```
        }
```

```
    } while (--N);
```

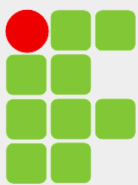
```
    queue->Q_ptr = Q_ptr;
```

```
}
```

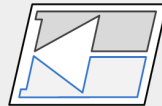
Função com 3 argumentos.

Uma instrução a + que a _v1

**Entretanto, o processamento
é mais eficiente.**

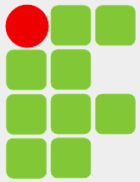


Divisões e Multiplicações

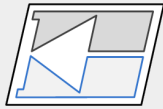


- Sempre que possível use operações inteiras e sem sinal. Se houver suporte do hardware à operações com ponto flutuante, considere o possível impacto no desempenho.
- Evite empregar divisões em HW sem suporte, nos que possuem HW para divisão, considere o possível impacto no desempenho*.
- Use simplificações matemáticas para resolver o problema e diminuir a carga computacional. Expanda o dado com a multiplicação, antes de comprimir com a divisão.
- Procure utilizar variáveis com o mesmo número de bits da CPU e cuidado com os truncamentos, *overflow* e *underflow*.

* No Cortex-M3 uma divisão pode levar de 2 a 12 ciclos de relógio.



Divisões e Multiplicações



Exemplo:

O LM35 apresenta uma saída de 10 mV/°C. O valor de leitura de um AD (10 bits), interno a um µC de 8 bits, é dado por $ADC = Vin \times 1024 / Vref$.

Como $Vref = 1,1V$, para converter o valor do AD para graus Celsius, é necessário multiplicar o valor ADC por 1100/1024 (considerando um dígito decimal antes da vírgula).

Utilizando a simplificação matemática e mantendo a variável **temp** com 16 bits, qual a operação matemática que deve ser executada?

$$temp = \mathbf{X} \times \frac{Vin \times 1024}{Vref}$$

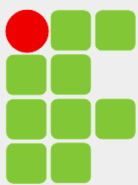
$$\frac{1100}{1024} = 1 + \frac{19}{256}$$

0,1°C → $temp = 1$

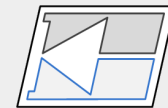
↓

$$temp = \frac{1100}{1024} \times \frac{0,001 \times 1024}{1,1}$$

$$temp = ADC + (ADC \times 19) / 256$$



Outros Comentários



- Use ponteiros sempre que possível.
- *Inline* funções críticas para o desempenho e com poucas linhas, não em *assembly*, porque o compilador não consegue otimizar adequadamente.

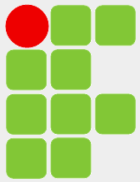
- Não escreva códigos “inteligentes” (não use compactação).

ex.: `if ( (a & 0x1FFFFFFF) !=0) b|=0x00000001;` ao invés de `b|=!! (a <<1) ;`

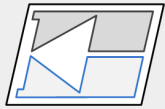
Do ponto de vista da manutenção, códigos **simples** são a melhor escolha. Em geral usar mascarará de bits e deslocamentos gera códigos mais confiáveis em compiladores diferentes.

A regra é: **se é fácil para algum outro programador entender, então será fácil de manter e compilará de forma eficiente também.**

- Cuidado com funções prontas e bibliotecas, elas podem gerar códigos extensos e ineficientes (ex. `printf()`).



Diretivas para o Compilador

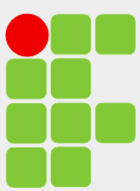


Para escrever códigos que são amigáveis ao compilador é preciso entender como o compilador trabalha. Isso é muito importante quando se deseja otimizar o código.

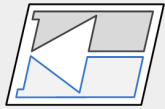
Sem nenhuma opção de otimização, o objetivo do compilador é reduzir o custo da compilação e fazer com que a depuração produza sempre os resultados esperados.

Exemplo de diferentes níveis de otimização para um determinado código.

Optimization flags	Size of .text	Time for test #1	Time for test #2
-O3	6898	903 μ s	19.7 ms
-O2	6666	972 μ s	20.1 ms
-Os	6618	955 μ s	20.1 ms



Diretivas para o Compilador

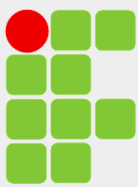


Diretiva de compilação para o gcc:

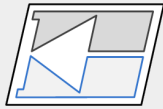
-On *// n é o nível de otimização*

None (-O0)
Optimize (-O1)
Optimize more (-O2)
Optimize most (-O3)
Optimize for size (-Os)

- O0** → Desabilita totalmente a otimização (*default*).
- O1** → A otimização leva mais tempo e consome mais memória para funções grandes.
- O2** → Otimiza mais. O GCC realiza quase todas as otimizações suportadas que não envolvam a relação memória/velocidade. O compilador não executa *loop unrolling* ou *inlining function*.
- O3** → Otimiza ainda mais. Habilita todas as otimizações de -O2 e também tenta fazer funções pequenas *inline* e renomear registradores.
- Os** → Otimização para tamanho. Habilita todas as otimizações de -O2 que normalmente não aumentam o tamanho do código. Também, realiza outras otimizações para reduzir o tamanho do código.



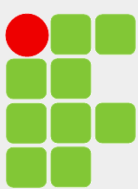
Algoritmos com FSM



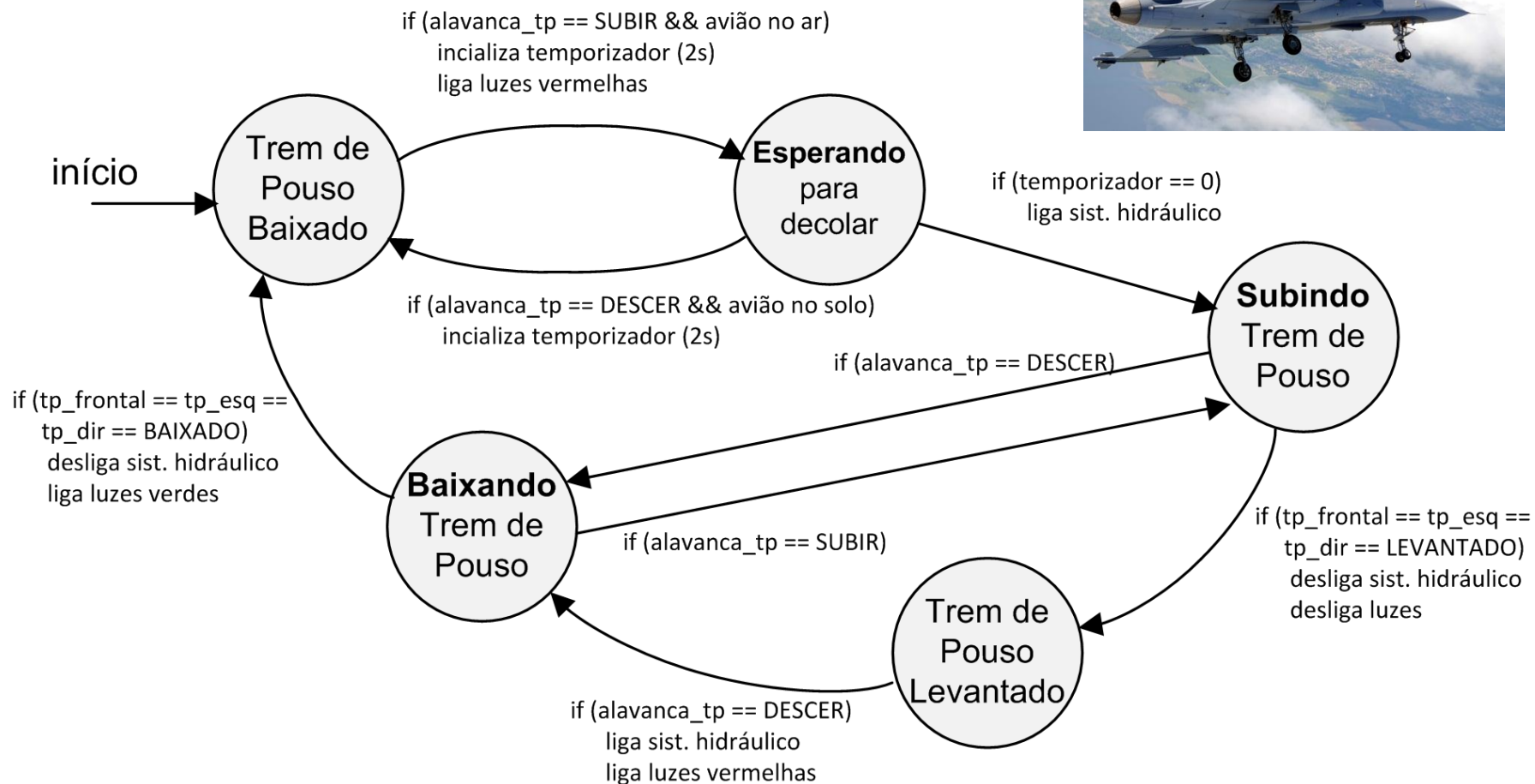
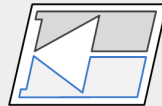
Muitos softwares para sistemas embarcados são candidatos a mecanização por uma máquina de estados. Um programa que deve seguir uma sequência de ações, lidar de forma diferente com suas entradas de acordo com diferentes modos de operação, geralmente é melhor programado usando uma máquina de estados.

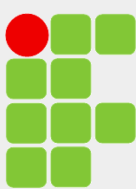
Para tal, os seguintes passos deve ser seguidos:

- Entender o que o usuário deseja.
- Desenhar o diagrama de transição de estados.
- Faça o esqueleto do código, sem os detalhes das transições.
- Garanta que as transições funcionem adequadamente.
- Faça os detalhes das transições.
- Teste.

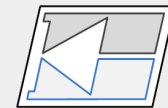


Exemplo – Trem de Pouso





Código Exemplo em C



```
typedef enum {TP_BAIXADO, ESP_DECOLAR, SUBINDO_TP,
              TP_LEVANTADO, BAIXANDO_TP} Tipo_Estado;

/* Tabela com o ponteiro para a chamada de funções de cada estado */
void (*tabela_estados[]= {trem_pouso_baixado, esperando_decolar, subindo_trem_pouso,
                          trem_pouso_levantado, baixando_trem_pouso })();

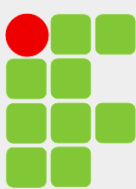
Tipo_Estado estado_atual;

main()
{
    inic_sistema();

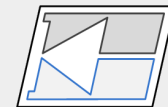
    /* A engrenagem principal da FSM se dá nesse laço. A função
    correspondente ao estado atual é chamada uma vez por iteração. */
    while(1)
    {
        tabela_estados[estado_atual]();
        decr_temporizador();

        /* faz outras coisas não relacionadas a esta FSM */
        ...
    }
}
//-----
void inic_sistema()
{
    estado_atual = TP_BAIXADO;
    inic_temporizador(0);

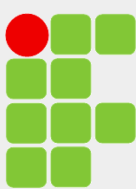
    /*desliga os demais mecanismos, luzes, etc. */
    ...
}
//-----
```



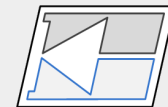
Código Exemplo em C



```
//-----  
void trem_pouso_baixado()  
{  
    /* eleva o trem de pouso sob comando, mas não se o avião está no solo */  
    if(alavanca_tp == SUBIR) && (anterior_alavanca_tp == DESCER) && (chave_solo == 0)  
    {  
        inic_temp(2);                // inicializa temporizador para 2 s (ele é decrementado)  
        estado_atual = ESP_DECOLAR;  
        liga_luzes_vermelhas();  
    }  
    anterior_alavanca_tp = alavanca_tp; // armazena o valor da alavanca do trem de pouso  
}  
//-----  
void esperando_decolar()                // já saiu do solo  
{  
  
    if( temporizador == 0)  
    {  
        estado_atual = SUBINDO_TP;  
        liga_sist_hidraulico();  
    }  
  
    /* se o avião toca o solo novamente, ou se o piloto muda de idéia, começa novamente */  
    if((alavanca_tp == DESCER)|| (chave_solo == 1))  
    {  
        inic_temp(2);  
        estado_atual = TP_BAIXADO;  
        anterior_alavanca_tp = DESCER;    // armazena ação do piloto  
    }  
}  
//-----
```

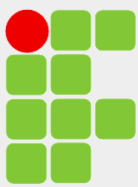


Código Exemplo em C

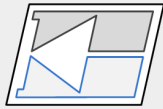


```
//-----  
void subindo_trem_pouso()  
{  
    /* quando os 3 trem de pouso estão levantados, vai para o estado TP_LEVANTADO */  
    if((tp_frontal == LEVANTADO) && (tp_esq == LEVANTADO) && (tp_dir == LEVANTADO))  
    {  
        estado_atual = TP_LEVANTADO;  
        desliga_luzes();  
        desliga_sist_hidraulico();  
    }  
  
    /* se o piloto muda de idéia */  
    if(alavanca_tp == DESCER)  
        estado_atual = BAIXANDO_TP;  
}  
//-----  
void trem_pouso_levantado()  
{  
    /* se o piloto move a alavanca do trem de pouso para baixo, baixa o trem de pouso */  
    if(alavanca_tp == DESCER)  
    {  
        estado_atual = BAIXANDO_TP;  
        liga_luzes_vermelhas();  
        liga_sist_hidraulico();  
    }  
}  
//-----
```

```
//-----  
void baixando_trem_pouso()  
{  
    if(alavanca_tp == SUBIR)  
        estado_atual = SUBINDO_TP;  
  
    if((tp_frontal == BAIXADO) && (tp_esq == BAIXADO) && (tp_dir == BAIXADO))  
    {  
        estado_atual = TP_BAIXADO;  
        desliga_sist_hidraulico();  
        liga_luzes_verdes();  
    }  
}  
//-----
```

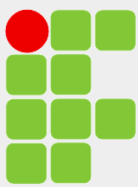


Considerações

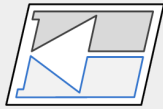


- Computadores são muito mais rápidos que o hardware mecânico – você pode ter que esperar.
- Como seu programa se comportará se alguma parte mecânica ou elétrica quebrar? Preveja testes no programa.
- Use ponteiro para funções versus `switch(case)`. É mais eficiente e confiável, gerando funções menos longas e mais fáceis de alterar.
- Na codificação da FSM tente manter a máxima correspondência entre o diagrama de estados e o programa. Se necessário, “esconda” os detalhes de hardware em outra camada de funções. A simetria com o diagrama de estados ajuda a evitar erros e é um dos motivos das máquinas de estado serem empregadas em sistemas embarcados.
- Inclua *prints* no programa para informar sobre o estado presente (entradas e saídas), o que leva a regra de **OURO** do teste de software”:

Não verifique apenas o que você quer que o programa faça, também verifique que ele não faça o que você não quer.

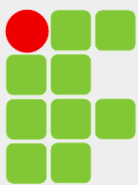


Watchdog Timer

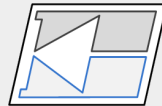


Problemas com WDT internos (μ C)

- Um estouro do WDT deve garantir uma inicialização do HW. Recarregar o contador de programa (PC) pode não inicializar adequadamente a CPU.
- WDT que gera uma interrupção não mascarável (NMI -*non-maskable-interrupt*) sem um *reset*, pode produzir uma inicialização inadequada de um sistema que falhou.
- Um WDT que se baseia em um pulso num pino de I/O não é muito seguro.
- Quando um par de escrita como 0x55 e 0xAA desabilita o WDT, algumas rotinas podem acidentalmente desabilitá-lo.
- Cuidado com WDT cujo registradores de controle possam ser escritos por uma falha de programação. O WDT pode ser desabilitado.
- Se o estouro do WDT não gerar um *reset* dos dispositivos ligados ao μ C, cada periférico deve ser inicializado após o estouro. Embora a CPU volte ao normal, os dispositivos confusos de I/O podem manter o sistema rodando inadequadamente.

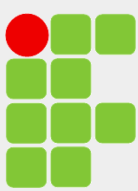


Watchdog Timer

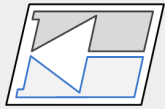


Característica para um ótimo WDT

- Não faça suposições sobre o estado do sistema após a inicialização do WDT. HW e SW podem estar confusos.
- O HW deve colocar o sistema em um estado seguro.
- Faça uma inicialização do HW após o estouro do WDT.
- Inicialize os periféricos também.
- Garanta que um programa falho não possa reprogramar o WDT.
- Deixe rastros para depuração após a inicialização por WDT.
- Coloque um *jumper* na PCI para desabilitar a depuração do WDT; remova ele quando o sistema for comercializado.
- Um ótimo WDT deve ser independente da CPU que quer proteger.



Watchdog Timer



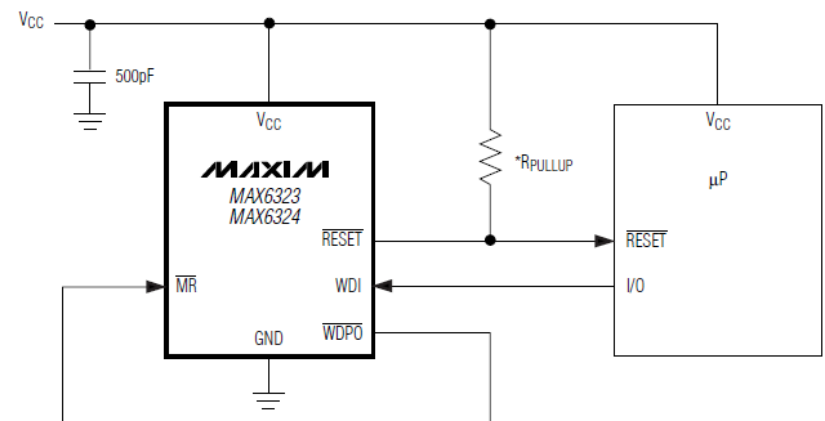
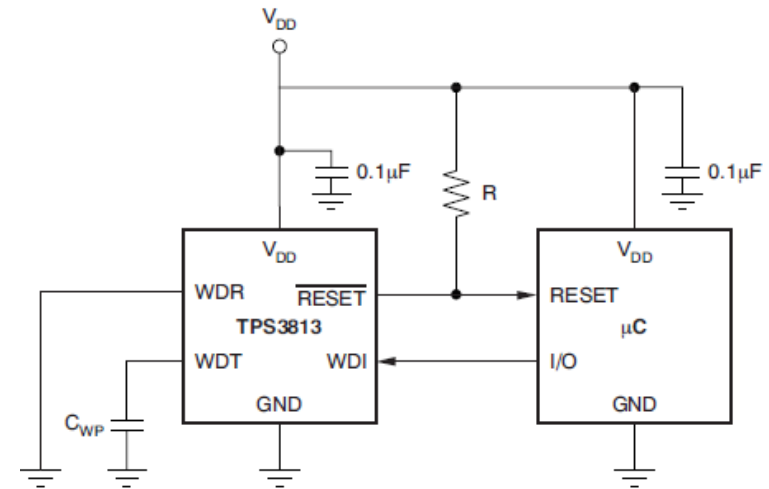
Pseudo-código para o tratamento de um WDT interno

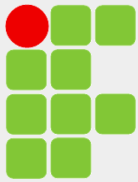
```
main()
{
    state = 0x5555;
    wdt_a();
    ...
    state += 0x2222;
    wdt_b();
}

wdt_a()
{
    if(state!=0x5555) halt;
    state += 0x1111;
}

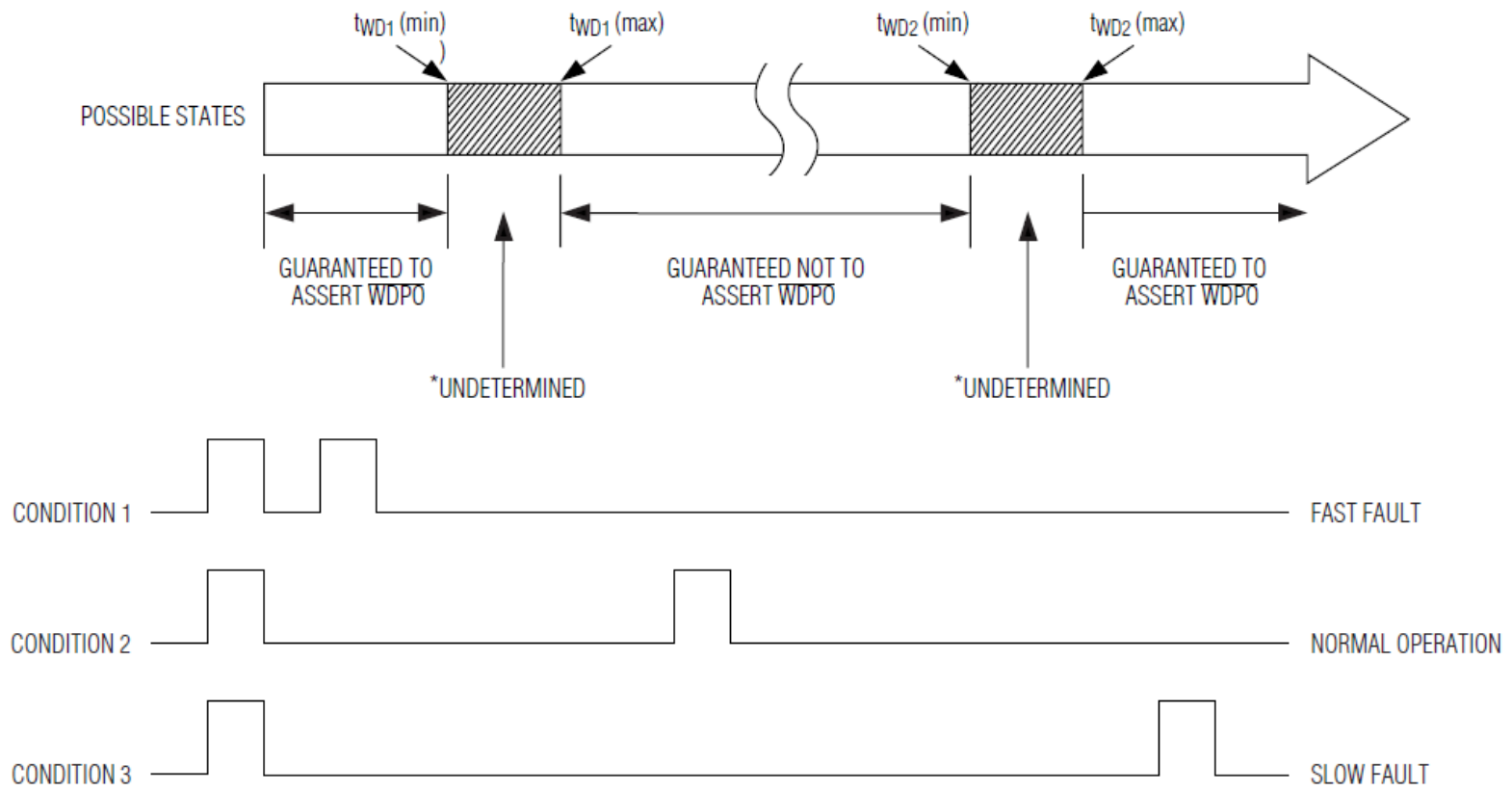
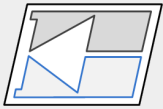
wdt_b()
{
    if(state!=0x8888) halt;
    state = 0;
    kick dog;
}
```

WDT externo

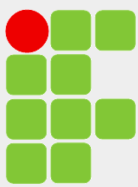




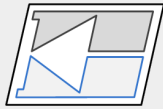
Watchdog Timer



*UNDETERMINED STATES MAY OR MAY NOT GENERATE A FAULT CONDITION.



Conclusão

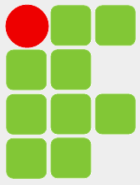


É fundamental seguir padrões de escrita de código para facilitar a manutenção, escrita e legibilidade do código. Sempre use boas ferramentas.

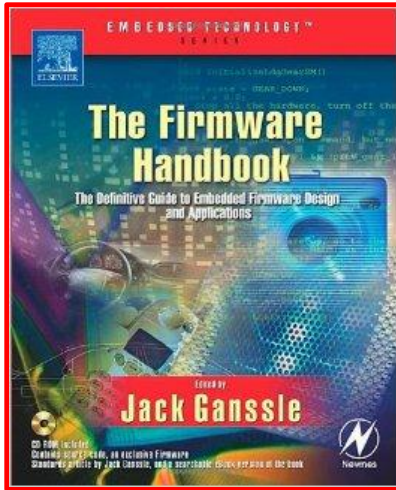
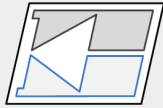
A otimização deve sempre que possível ser feita, melhorando o desempenho e consumo de memória, o que também produz códigos mais robustos. É importante compreender como o compilador funciona e como pode ser ajustado para realizar otimizações. É imprescindível a análise do arquivo gerado na compilação (*map file*) fornecendo informações das funções mais utilizadas, tamanho de código, etc.

Muitos sistemas embarcados rodam por anos sem reiniciar. Produtos confiáveis nunca devem conter código frágil. O desafio do projetista é identificar as questões delicadas e criar alternativas para que os sistemas funcionem sempre, todo o tempo.

Sistemas críticos, que não podem e que devem ser capazes de se recuperar de falhas, devem possuir um sistema de inicialização confiável.

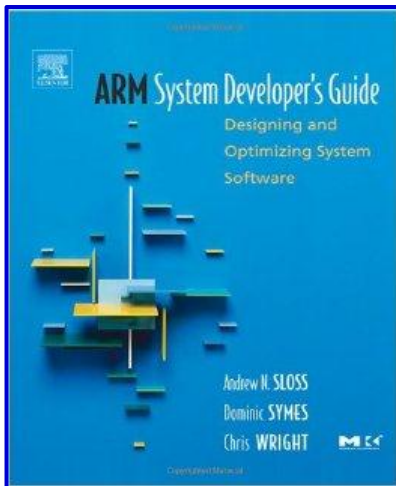


Bibliografia



GANSSLE, J. *The Firmware Handbook: The Definitive Guide to Embedded Firmware Design and Applications*. 1a ed. Elsevier, 2004.

SLOSS, Andrew N, Dominic Symes and Chris Wright. *ARM System Developer's Guide: Designing and Optimizing System Software*. 1ed. Elsevier. 2004.



C Style Guide – NASA, 1994.

Cortex M3 Reference Manual – ARM, revisão r1p1, 2005.

MAX6323, TPS3813 – *datasheet*.

Leituras obrigatórias!